Lecture 2: Large Language Models Architecture and Practice

Juan F. Imbet

Paris Dauphine – PSL University

Large Language Models in Finance

Outline

From Words to Documents: Three Aggregation Strategies

Why documents? Earnings calls ≈10 k tokens. 10-K filings ≈100 k tokens.

Method	**Formula**	**Weakness**
Mean embedding	$\bar{\varphi}(d) = \frac{1}{n}\sum_{i=1}^{n} \varphi(w_i)$	No word order
TF-IDF weighted	$\varphi_{\text{tfidf}} = \frac{\sum \text{tfidf}(w_i)\,\varphi(w_i)}{\sum \text{tfidf}(w_i)}$	Still bag-of-word
SBERT (contextual)	Siamese BERT fine-tuned on NLI	Compute at enc

Key insight

Mean embedding ignores word order. "Revenue rose, costs fell" ≡ "Costs rose, revenue fell" under mean pooling.

SBERT advantage: pre-compute once; search is $O(1)$ at query time.

RNNs and the Vanishing Gradient Problem

Vanilla RNN recurrence:

$$\mathbf{h}_t = \sigma(W_h\,\mathbf{h}_{t-1} + W_x\,\mathbf{x}_t + \mathbf{b})$$

BPTT gradient:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \sum_{s=t}^{T} \frac{\partial \ell_s}{\partial \mathbf{h}_s} \prod_{k=t}^{s-1} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k}$$

UNDER THE HOOD

Proposition (Vanishing & Exploding Gradients). Let $\rho = \|W_h\|_2$, $M_\sigma = \sup_z |\sigma'(z)|$. Then

$$\left\| \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k} \right\|_2 \leq (M_\sigma\,\rho)^{T-t}$$

For logistic sigmoid: $M_\sigma = \frac{1}{4}$. If $M_\sigma \rho < 1$: gradients vanish exponentially.

Finance implication: An earnings call of 10 k tokens makes long-range dependencies (page 3 $\leftrightarrow$ page 20) practically unlearnable.

LSTM: Six Equations, One Key Idea

The six equations:

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{forget gate}$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{input gate}$$

$$\mathbf{g}_t = \tanh(W_g[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_g) \quad \text{candidate cell}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad \text{cell state (additive!)}$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{output gate}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{hidden state}$$

UNDER THE HOOD

Why the cell state avoids vanishing gradients.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t+1}} \odot \mathbf{f}_{t+1} + (\text{terms via } \mathbf{h}_t)$$

When $\mathbf{f}_{t+1} \approx \mathbf{1}$: gradient flows back undamped.
(*"Constant error carousel"* — Hochreiter & Schmidhuber, 1997)

Bahdanau Attention: The Bridge to Transformers

The bottleneck: encoder must compress the full input into one fixed vector.

Solution (Bahdanau et al., 2015): let the decoder attend to all encoder states.

Alignment score:

$$e_{t,i} = \mathbf{v}^\top \tanh(W_s \mathbf{s}_t + W_h \mathbf{h}_i)$$

Bahdanau Attention (cont.)

Attention weights and context vector:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \qquad \mathbf{c}_t = \sum_i \alpha_{t,i}\, \mathbf{h}_i$$

$\{\alpha_{t,i}\}$ form a probability distribution over encoder positions.

THE BIGGER PICTURE

Decoder generating a sentence about *net income* assigns high $\alpha_{t,i}$ to the tokens in the transcript where income figures appear. Attention is interpretable.

Next step: apply attention *within* a single sequence $\rightarrow$ self-attention $\rightarrow$ Transformer.

Scaled Dot-Product Attention

Given input matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$, project to queries, keys, values:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

UNDER THE HOOD

Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $QK^\top \in \mathbb{R}^{n \times n}$: entry $(i, j) = \mathbf{q}_i^\top \mathbf{k}_j$
- Row-wise softmax $\rightarrow$ attention weights $A_{ij} > 0$, $\sum_j A_{ij} = 1$
- Output for token i: $\sum_j A_{ij}\, \mathbf{v}_j$ (convex combination of values)

Causal mask (decoder): set $\tilde{S}_{ij} = -\infty$ for $j > i$ to prevent attending to future tokens.

Multi-Head Attention

Run h attention heads in parallel, each with its own projections:

$$\text{head}_i = \text{Attention}(XW_i^Q,\, XW_i^K,\, XW_i^V)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

Original Transformer: $h = 8, \quad d_k = d_v = 64, \quad d_{\text{model}} = 512.$

What do heads learn?

- Lower layers: syntactic dependency (subject–verb agreement, negation scope)
- Deeper layers: semantic associations (entity co-reference, temporal relations)
- Financial models: heads specialize on "last quarter" $\rightarrow$ figures, entity linking

Complexity: $O(n^2 d_{\text{model}})$ per layer. For $n = 4{,}096$ tokens: $\approx$134M values per layer. Motivates FlashAttention (Dao et al., 2022).

The Full Encoder Block

Two sub-layers with residual connections and Layer Normalisation:

$$Z^{(\ell)} = \text{LayerNorm}\Big(X^{(\ell)} + \text{MultiHead}(X^{(\ell)})\Big)$$

$$X^{(\ell+1)} = \text{LayerNorm}\Big(Z^{(\ell)} + \text{FFN}(Z^{(\ell)})\Big)$$

where the feed-forward network is:

$$\text{FFN}(\mathbf{z}) = \max(0,\, \mathbf{z}W_1 + \mathbf{b}_1)\, W_2 + \mathbf{b}_2, \quad d_{\text{ff}} = 4\, d_{\text{model}}$$

Residual connections are critical

Direct gradient pathways from output to input eliminate the multiplicative vanishing-gradient problem across layer depth.

Scale: base Transformer: $L = 6$ layers. Modern LLMs: $L = 32$–96 layers, $d_{\text{model}} = 4{,}096$–$8{,}192$.

BERT: Masked Language Modelling

BERT = Bidirectional Encoder Representations from Transformers (Devlin et al., 2019).

MLM objective: mask 15% of tokens; predict originals.

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log p_\theta\left(w_i \mid \mathbf{x}_{\setminus \mathcal{M}}\right)$$

Masking protocol:

- 80%: replaced with `[MASK]`
- 10%: replaced with a random token
- 10%: left unchanged (forces the model to build a representation for every token)

Architecture: encoder-only, fully bidirectional attention.

THE BIGGER PICTURE

Best for: classification (sentiment, credit rating), token-level extraction (NER, metric extraction), semantic similarity. **Not for** generation.

GPT: Causal Language Modelling and Architecture Choice

CLM objective: predict the next token from all preceding tokens.

$$\mathcal{L}_{\text{CLM}}(\theta) = -\sum_{t=1}^{T} \log p_\theta(w_t \mid w_1, \ldots, w_{t-1})$$

Causal mask ensures $j \leq i$ only. Natural autoregressive generation.

Task	**Architecture**	**Why**
Sentiment classification	Encoder (BERT)	Full bidirectional context
NER in filings	Encoder (BERT)	Token-level labels
Earnings call summarisation	Enc–Dec (T5)	Seq2seq; full input read
Report drafting, Q&A	Decoder (GPT)	Generative by design
Numerical reasoning (FinQA)	Decoder + CoT	Multi-step generation

Reasoning models (o1, o3, DeepSeek-R1)

Trained via RL with verifiable rewards to produce chain-of-thought traces. Same decoder architecture; different training regime. Outperform standard generation on FinQA-type tasks; do not eliminate hallucination.

The Modern LLM Landscape

Model	Org.	Arch.	Params	Context	Finance
GPT-4o	OpenAI	Decoder	$\sim$1T	128K	No
Claude 3.x/4.x	Anthropic	Decoder	n/d	200K	No
Llama 3 (405B)	Meta	Decoder	405B	128K	No
Gemini 1.5 Pro	Google DM	Decoder	n/d	1M	No
FinBERT	Araci 2019	Encoder	110M	512	Yes
BloombergGPT	Bloomberg	Decoder	50B	2K	Yes

Key findings:

- FinBERT: +8 pp over BERT-base on FinancialPhraseBank (all-agree split)
- BloombergGPT: trained on 363B finance + 345B general tokens
- General frontier models at large scale often *match* BloombergGPT — scale compensates for domain specificity
- Open-weight models (Llama) enable on-premises deployment: no data-privacy risk

Sampling Strategies: Temperature, Nucleus, Beam

Temperature scaling:

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

$\tau \to 0$: greedy (deterministic). $\tau = 1$: standard. $\tau \to \infty$: uniform.

Nucleus (top-p) sampling:

$$\mathcal{V}_p = \arg\min_S \left\{ |S| : \sum_{i \in S} p_i(\tau) \geq p \right\}$$

Adapts vocabulary size to model confidence. Common defaults: $p = 0.9$–0.95.

Beam search: keep B best partial sequences at each step; expand all.

- Pros: higher log-probability sequences; good for ROUGE/BLEU tasks
- Cons: beam search degeneration for open-ended generation

Sampling Strategy: Finance-Specific Guidance

Task	Temperature	Sampling
Structured data extraction (earnings, covenants)	$\tau = 0$ (greedy)	—
Sentiment labelling	$\tau \approx 0.1$	greedy / top-k
Document summarisation	$\tau \approx 0.3$–0.5	top-$p = 0.9$
Regulatory Q&A / compliance	$\tau \approx 0.1$	greedy + RAG
Scenario / stress-test narrative	$\tau \approx 0.7$–1.0	top-$p = 0.95$

Rule of thumb

If the output feeds a downstream system or will be compared to ground truth: $\tau = 0$.
If a human analyst values breadth and novelty: $\tau \in [0.5, 1.0]$ with nucleus.

Self-consistency: run K samples at $\tau > 0$; take majority vote. Especially effective for multi-step numerical reasoning (FinQA-type tasks).

Structured Generation: Validity Mask and Pydantic

Validity mask:

$$M_t[i] = \mathbf{1}[\text{token } i \text{ can extend } x_{<t} \text{ to a string in } \mathcal{L}(\mathcal{G})]$$

Invalid tokens receive $-\infty$ log-probability *before* softmax.

Anthropic tool use + Pydantic:

```
class EarningsReport(BaseModel):
    revenue_bn: Optional[float] = None
    net_income_bn: Optional[float] = None
    eps: Optional[float] = None

client.messages.create(
  tools=[{"name": "record_earnings",
          "input_schema": EarningsReport.model_json_schema()}]
  tool_choice={"type": "tool", "name": "record_earnings"},
  ...
)
```

Structured Generation: Failure Modes

Three failure modes even with constrained decoding:

1. Numerically wrong-but-valid output (pair with $\tau = 0$ + RAG)
2. Optional fields $\rightarrow$ explicit `null` (detectable)
3. Unit confusion $\rightarrow$ application-level validation required

API Mechanics: BPE Tokenisation and Cost Model

BPE tokenisation: iteratively merge the most frequent adjacent symbol pair.

- Vocabulary: 32k–100k sub-words
- Rule of thumb: 1 English word $\approx$ 1.3 tokens
- "EBITDA" $\rightarrow$ `["EB", "IT", "DA"]`; rare tickers get more fragments

Cost model:

$$\text{cost} = N_{\text{in}} \cdot c_{\text{in}} + N_{\text{out}} \cdot c_{\text{out}}$$

Model	**Input ($/MTok)**	**Output ($/MTok)**
GPT-4o	\$5.00	\$15.00
Claude 3 Haiku	\$0.25	\$1.25
Llama 3 self-hosted	$\approx$\$0	$\approx$\$0

Worked example: 10 k transcripts $\times$ 7 k input tokens + 400 output tokens:

$$\text{cost}_{\text{GPT-4o}} = 10{,}000 \times (0.035 + 0.006) = \$410$$

Same workload on Claude Haiku: $\approx$\$20 (20$\times$ cheaper).

RAG Pipeline: Three-Stage Definition

Retrieval-Augmented Generation (RAG):

1. **Retrieval:** score passages against query q; return top-k passages $\mathcal{R}(q)$
2. **Augmentation:** construct augmented prompt $\tilde{q} = [d_{(1)}, \ldots, d_{(k)}, q]$
3. **Generation:** $a \sim P_\theta(\cdot \mid \tilde{q})$

THE BIGGER PICTURE

Why RAG for finance?

- Knowledge base updated by adding new filings — no retraining
- Every factual claim traceable to a retrieved passage (auditability)
- Dominant architecture for document Q&A in production financial systems

FinanceBench lesson (Zhang et al., 2024)

GPT-4-Turbo with a retrieval system *incorrectly answers or refuses* 81% of questions from SEC filings. Naive RAG is insufficient for high-stakes

Hallucination Taxonomy and Detection

Definition: output that is fluent, confident, and not supported by the input or the external world.

Three sub-types for finance:

1. Numerical hallucination: D/E ratio 1.8 $\rightarrow$ reported as 0.8
2. Entity hallucination: wrong company / regulatory body
3. Citation hallucination: fabricated regulation, ruling, or standard

Detection methods:

- **SelfCheckGPT:** draw N samples; hallucinated claims are inconsistent across samples
- **FActScore:** decompose into atomic claims; verify each against retrieval
- **Inner confidence** (Chen et al., 2024): high-confidence LLM signals achieve Sharpe +20% vs. unconditional benchmark

Root cause

Models are trained to produce plausible text. $\tau = 0$ removes stochasticity but does **not** eliminate hallucination — the most probable completion

Hallucination Mitigation: Five-Layer Stack

These techniques are complementary — use them together in production.

1. **Retrieval-augmented generation (RAG):** ground context in verified passages; model almost never fabricates figures absent from retrieved text
2. **Self-consistency sampling:** K independent chains; majority-vote answer. Best for multi-step numerical reasoning
3. **Chain-of-thought prompting:** require explicit intermediate steps; each step is independently verifiable
4. **Claim decomposition (FActScore):** break output into atomic claims; flag unsupported / contradicted claims before human review
5. **Confidence elicitation + abstention:** prompt the model to flag uncertainty; calibrate thresholds against accuracy curves

Reasoning models

Reduced hallucination on tasks with verifiable intermediate steps (FinQA). But extrinsic hallucination on obscure regulatory facts persists. All five techniques remain necessary.

Responsible Use: Regulation, Look-Ahead Bias, GDPR

EU AI Act (Regulation 2024/1689, in force August 2024):

- High-risk categories: credit scoring, insurance pricing, AI-assisted securities pricing
- Requirements: conformity assessments, human oversight, transparency to regulators

Look-ahead bias (Didisheim et al., 2025):

- LLMs can reconstruct historical aggregate time series from parametric memory
- Produces spurious return predictability in back-tests
- Any information the model might have memorised is contaminated data

GDPR: fine-tuning or RAG on client personal data triggers right-to-erasure obligations. A model trained on personal data cannot satisfy erasure without retraining.

Geographic bias: all five major benchmarks are North American / English. Sentiment models trained on US earnings calls may systematically

Lecture 2 Summary

Architecture arc:

- Mean / TF-IDF / SBERT document embeddings
- RNN vanishing gradient $\rightarrow$ LSTM cell state (additive update) $\rightarrow$ GRU
- Bahdanau attention $\rightarrow$ self-attention $\rightarrow$ Transformer
- Scaled dot-product attention, $\sqrt{d_k}$ proof, multi-head attention, sinusoidal PE + rotation
- Full encoder block: residual + LayerNorm

Pre-training and LLMs: BERT (MLM, bidirectional) vs. GPT (CLM, causal) · reasoning models · FinBERT · BloombergGPT · five benchmarks

Lecture 2 Summary (cont.)

Engineering practice: Temperature / nucleus / beam · finance guidance table · validity mask · Pydantic / tool use · BPE · cost model · prompt caching · RAG (dense + BM25 + RRF + re-rank) · FinanceBench
Safety: Hallucination taxonomy · SelfCheckGPT · FActScore · 5-layer mitigation · EU AI Act · look-ahead bias · GDPR · geographic bias

Next lecture

Lecture 3: Sentiment Analysis in Finance — applying these foundations to FinancialPhraseBank, FinBERT fine-tuning, and LLM-based sentiment extraction.

Appendix

Extended Derivations, Encodings, and Benchmarks

Extra / optional material — beyond the core lecture.

Why Divide by $\sqrt{d_k}$? The Variance Proof

Setup: suppose $q_\ell, k_\ell \overset{\text{i.i.d.}}{\sim} (0,1)$ for $\ell = 1, \ldots, d_k$.

Step 1. Compute the variance of the dot product:

$$\text{Var}\left(\mathbf{q}_i^\top \mathbf{k}_j\right) = \sum_{\ell=1}^{d_k} \text{Var}(q_\ell k_\ell) = d_k$$

Step 2. For large d_k, logits concentrate near $\pm\sqrt{d_k}$:

$$\text{softmax gradient:} \quad \sigma(z)(1-\sigma(z)) \approx 0 \text{ for large } |z|$$

Step 3. Divide by $\sqrt{d_k}$ to normalise variance to 1:

$$\text{Var}\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}\right) = 1$$

Consequence

Without scaling, the softmax saturates and gradients vanish — we reintroduce the same problem LSTMs were designed to fix, now inside the attention layer.

Positional Encoding and the Rotation Property

Self-attention is permutation-equivariant $\Rightarrow$ position must be injected.

Sinusoidal positional encoding:

$$\mathrm{PE}_{(\mathrm{pos},2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d}}\right), \quad \mathrm{PE}_{(\mathrm{pos},2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d}}\right)$$

Small i: high frequency (fine position). Large i: low frequency (coarse position).

Rotation property: for any fixed offset k,

$$\begin{pmatrix} \mathrm{PE}_{(\mathrm{pos}+k,2i)} \\ \mathrm{PE}_{(\mathrm{pos}+k,2i+1)} \end{pmatrix} = \begin{pmatrix} \cos\omega_i k & -\sin\omega_i k \\ \sin\omega_i k & \cos\omega_i k \end{pmatrix} \begin{pmatrix} \mathrm{PE}_{(\mathrm{pos},2i)} \\ \mathrm{PE}_{(\mathrm{pos},2i+1)} \end{pmatrix}$$

The rotation matrix depends only on k, not on the absolute position $\Rightarrow$ **relative positions are linearly decodable**.

Modern variant: **RoPE** (Rotary Position Embeddings) embeds the rotation directly into query–key dot products. Used in Llama, Mistral, Qwen.

Financial NLP Benchmarks

Benchmark	Task	Size	Metric
FinancialPhraseBank	Sentiment (3-class)	4,840 sent.	Accuracy
FiQA SA / QA	Opinion mining; Q&A	∼1,200	F1, NDCG
ECTSum	Earnings call summarisation	2,425 docs	ROUGE-L
FinQA	Numerical reasoning	8,281 Q&A	Exec. acc.
FLUE	Multi-task financial NLU	Multi-task	Task-specific

Geographic bias

All five benchmarks are built from English-language, North American sources. Do not assume generalisation to European or Asian financial contexts.

FinQA requires multi-step arithmetic over tables — reasoning models dominate.
ECTSum evaluates abstractive summarisation with expert-written gold summaries.

Dense, Sparse, and Hybrid Retrieval

Dense retrieval:

$$s_{\text{dense}}(q, d_i) = \frac{\phi(q) \cdot \phi(d_i)}{\|\phi(q)\| \|\phi(d_i)\|}$$

SBERT bi-encoder; FAISS / HNSW for ANN search.

BM25 (sparse lexical): Strong on exact matches (CUSIP numbers, filing section references, tickers).

Hybrid via Reciprocal Rank Fusion:

$$\text{RRF}(d) = \sum_{r \in \{\text{dense}, \text{sparse}\}} \frac{1}{60 + \text{rank}_r(d)}$$

Robust to score-scale differences; no learned weighting parameter.

Method	**Strength**	**Weakness**
Dense	Semantic similarity	Exact-match failures
BM25	Exact-match recall	Misses paraphrases
Hybrid	Both	Slightly slower